

Ejercicio 1

Descargar este enunciado [pdf](#) o [html](#)

Partiendo del proyecto en [Vehiculos_recurso](#) que contiene ya ciertos módulos y clases que nos van a ser útiles para la realización del examen y entre los cuales **podemos destacar**:

En el paquete `com.vahiculos.data` :

- En el paquete `.coche` están `CocheMock.kt` y `CocheDaoMock.kt` que contiene un listado de coches que vamos a utilizar para la primera carga de datos en la base de datos de Room.

En el paquete `com.vehiculos.ui.features` :

- Puedes definir si quieres `CocheUiState.kt` para guardar el estado de la ficha de un coche y de los items de la lista. Pero en cualquier caso, el modelo de datos que debes usar para la UI es el mismo que el del dominio, es decir, el modelo `Coche` que se encuentra en el paquete `com.vehiculos.models` . Por tanto, **puedes usar directamente esta clase**.
- Em el paquete `.fichacoche` dispones de `FichaCocheScreen.kt` que contiene la implementación de la UI de la ficha de un coche y que deberás completar según las especificaciones.
- Em el paquete `.galeriacoches` dispones de `ItemListaCoches.kt` que contiene la definición de la UI para ver un coche en la lista a modo de galería de coches y que puedes ver en la imagen de ejemplo justo abajo.



Volkswagen Passat 2.0 TDI 150CV
BMT Advance DSG 4p

Año: 2016

34568 €
antes 43210 €



BMW
X5

65100 €
antes 86800 €

25%



Tip

Todas las imágenes de coches están disponibles en la carpeta `recursos\galeria_coches` .

Copialá en la raíz del servidor Nginx. Fíjate que las URL en el mock de carga la url de la foto de los coches es `http://10.0.2.2/galeria_coches/foto_X.png` .

El resto de imágenes o iconos los puedes encontrar en la carpeta `res/drawable` del proyecto.

Especificaciones

1. Crea un BD con Room para almacenar todos los datos de nuestro modelo denominada "**coches.db**". La BD debe contener una única tabla denominada **coches** donde la clave primaria será el campo **id** de tipo **Int** que tienes en el modelo. Los campos de las tabla deben ir en `snake_casing`

Las operaciones sobre la tablas serán todas asíncronas y debes definir las siguientes:

- Operaciones básicas `insert` , `delete` , `update` y `count`
- `get()` : Devuelve todos los coches de la base de datos.
- `get(Int)` : Devuelve un coche de la base de datos a partir de su id.
- `getOertas()` : Devuelve todos los coches de la base de datos que tengan un porcentaje de descuento mayor que 0.

```
SELECT * FROM coches WHERE porcentaje_descuento > 0
```
- `getOrdenadosPrecio()` : Devuelve todos los coches de la base de datos ordenados por precio de menor a mayor.

```
SELECT * FROM coches ORDER BY precio ASC
```
- `getOertasOrdenadasPrecio()` : Devuelve todos los coches de la base de datos que tengan un porcentaje de descuento mayor que 0 ordenados por precio de menor a mayor.

2. Añade las **anotaciones de Hilt en la aplicación y prepara los 'providers'** que consideres necesarios de las instancias de Room para la inyección de dependencias con **Hilt**. Teniendo en cuenta que se creará una **instancia única** de cada uno de ellos.

3. Modifica el '*repositorio*' para usar los métodos definidos.

4. Realiza la primera carga de datos de la base de datos de Room a partir de los datos

`CocheDaoMock` usando el insert del repositorio. **Solo si la BD está vacía.**

Para ello, realiza la carga de datos en la BD al iniciar la aplicación. Utilizando el método `onCreate` de la clase `VehiculosApplication` .



Nota

Recuerda que los métodos de Room deben ser asíncronos y que debes tener en cuenta que la carga de datos se realiza solo si la BD está vacía.

5. Por último, comprueba con el App Inspection que la carga de datos se ha realizado correctamente.

